TideWAL: Timestamp-Integrated Dependency-Closed Parallel WAL for Main-Memory Transaction Processing

Shun Sasaki[1] Takayuki Tanabe[1] Hideyuki Kawashima[1]

概要: This paper proposes TideWAL, a dependency-closed parallel WAL protocol for main-memory transaction processing. TideWAL avoids conservative global-prefix durable acknowledgment while preserving recovery safety by acknowledging a transaction only after both its own log record and the durable frontier of its observed dependencies are persistent. In an ERMIA-style implementation, TideWAL reduces pending durable commits from 65,522 to 48 and p99 durable-acknowledgment latency from 131 ms to 512 μs on YCSB-B at 32 threads, while sustaining 231K durable acknowledgments per second.

1. Introduction

1.1 Motivation

Transaction processing systems must provide isolation and durability under concurrent access. Serializability has long served as a fundamental correctness criterion for concurrent transaction execution [1], [2], while write-ahead logging (WAL) remains the standard mechanism for crash durability and recovery [3], [4].

Modern main-memory transaction processing systems, such as Silo, ERMIA, TicToc, and related designs, scale concurrency control by avoiding centralized bottlenecks in validation, timestamp management, and version management [5], [6], [7], [8]. In timestamp-based engines, committed transactions are already assigned commit timestamps that determine their logical serialization order. Consequently, once concurrency control scales, the commit-path bottleneck often shifts from transaction execution to durable acknowledgment.

A conventional centralized WAL provides a simple durability order: log records are appended to a single stream, and transactions are acknowledged according to a durable prefix of that stream. This design is safe, but it introduces a serialization point for log insertion and acknowledgment. Parallel WAL (P-WAL) removes this obvious bottleneck by partitioning log records across multiple WAL shards [9], [10], [11]. However, parallel log insertion alone is insufficient. The system must still decide when a transaction can be safely acknowledged as durable. In asynchronous commit pipelines, an overly conservative rule may preserve throughput only by accumulating a large backlog of pending durable commits, thereby increasing tail latency.

Thus, scalable durability should be evaluated not only by transaction execution throughput, but also by how quickly and safely the system returns durable commit acknowledgments.

1.2 Problem

Existing P-WAL designs often reintroduce a conservative global order at the durability layer. Even when the concurrency-control layer already assigns commit timestamps, the WAL layer may allocate a separate global log sequence number (LSN). A transaction is then acknowledged only when a global durable prefix reaches its LSN.

This rule is safe but overly conservative. It couples the acknowledgment of transactions that may be unrelated in the serialization order. If one WAL shard is slow, transactions on other shards may be delayed even when they have no dependency on transactions logged by the slow shard. With a bounded asynchronous pipeline, this delay may be hidden as high durable-acknowledgment throughput, but

[1] Faculty of Environment and Information Studies, Keio University, Japan

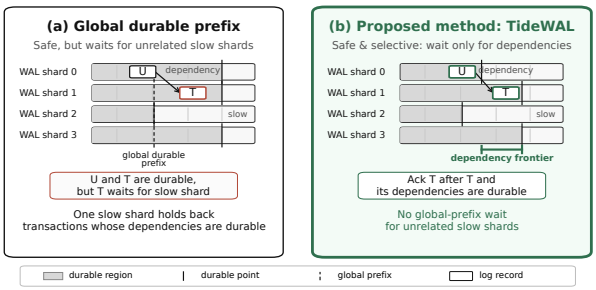

図 1 Durable acknowledgment policies in parallel WAL. Global-prefix acknowledgment is safe but conservatively waits for unrelated shards. Local-only acknowledgment avoids unrelated waits but is unsafe under dependencies. TideWAL acknowledges a transaction only after its own log record and its dependency frontier are durable.

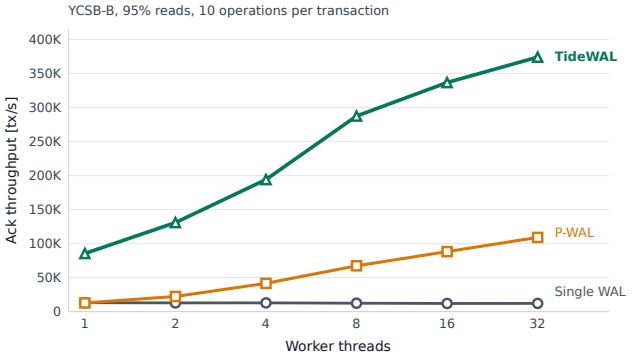

図 2 Durable acknowledgment behavior on ERMIA with YCSB-B at 32 threads. High durable-acknowledgment throughput under an asynchronous global-prefix policy can hide a large pending-commit backlog. TideWAL reduces both backlog and p99 durable-acknowledgment latency by using dependency-closed acknowledgment.

only by increasing the number of pending commits and tail latency.

The opposite design point is local-only acknowledgment: a transaction is acknowledged as soon as its own WAL shard becomes durable. This avoids waiting for unrelated shards, but it is unsafe in general. If transaction T reads a value written by transaction U, acknowledging T before U is recoverable may allow a crash recovery state that contains T without the state on which T depends.

Figure 1 summarizes these alternatives. Global-prefix acknowledgment is safe but may wait for unrelated slow shards. Local-only acknowledgment avoids such waits but can acknowledge a transaction before its dependencies are recoverable. The desired point is dependency-closed acknowledgment: acknowledge a transaction after its own log record and the transactions it depends on are durable.

This distinction is observable in our ERMIA implementation. Figure 2 shows YCSB-B [12] at 32 threads. An asynchronous global-LSN-prefix design achieves high durable-acknowledgment throughput, but accumulates 65,522 pending commits and increases p99 durable-acknowledgment latency to 131 ms. In contrast, TideWAL reduces pending commits to 48 and p99 latency to 512 μs by acknowledging transactions using dependency-closed durable frontiers.

1.3 Approach

This paper proposes *TideWAL*, a timestamp-integrated, dependency-closed parallel WAL protocol for main-memory transaction processing. TideWAL targets timestamp-based engines such as ERMIA, where the transaction engine already computes a logical commit order. Its goal is not merely to maximize raw throughput, but to return durable commit acknowledgments safely without the backlog and tail latency caused by global-prefix waiting.

TideWAL is based on three ideas.

First, TideWAL replaces global-prefix acknowledgment with dependency-closed durable acknowledgment. For each transaction, TideWAL tracks a dependency frontier that summarizes the transactions whose effects must be recoverable before the transaction itself can be acknowledged. A transaction is acknowledged only when its own log record and this dependency frontier are durable. This condition preserves recovery safety while avoiding waits for unrelated WAL shards.

Second, TideWAL reuses the commit timestamp as the logical LSN. Existing P-WAL designs may allocate a separate global LSN even when the transaction engine already assigns commit timestamps. TideWAL removes this duplicated ordering mechanism. WAL shards still maintain local physical offsets, but the WAL layer does not allocate an additional global logical order.

Third, TideWAL decouples execution from durable acknowledgment using a worker/flusher/committer pipeline. Workers execute and validate transactions, assign commit timestamps, and enqueue log records. Flushers persist shard-local log batches. Committers observe durable-frontier advances and acknowledge transactions whose dependency conditions are satisfied. Thus, workers do not directly block on `fdatasync` or on conservative durable-prefix waits.

We implement TideWAL in an ERMIA-style engine and evaluate it on YCSB workloads. On YCSB-B at 32 threads, worker-wait global-prefix WAL reaches only 106K durable acknowledgments per second. An asynchronous

global-prefix design improves throughput to 365K acknowledgments per second, but at the cost of 65,522 pending commits and 131 ms p99 durable-acknowledgment latency. TideWAL sustains 231K durable acknowledgments per second while reducing pending commits to 48 and p99 latency to 512 μs. These results show that dependency-closed acknowledgment controls durable-acknowledgment backlog and tail latency while preserving recovery safety on sparse-dependency workloads.

We also evaluate timestamp integration. Under real I/O, storage synchronization dominates, so the difference between separate-LSN logging and TideWAL is small. Under an I/O-light configuration, however, reusing commit timestamps as logical LSNs improves throughput by 7.4% by eliminating WAL-side global LSN allocation. This result shows that TideWAL removes duplicated ordering between concurrency control and durability, and that the benefit becomes visible once the storage bottleneck is weakened.

1.4 Organization

The rest of this paper is organized as follows. Section 2 reviews WAL, parallel WAL, timestamp-based concurrency control, and dependency requirements for recovery. Section 3 presents TideWAL's timestamp-integrated logical ordering, dependency-frontier construction, and worker/flusher/committer pipeline. Section 4 discusses the correctness of dependency-closed durable acknowledgment. Section 5 evaluates durable-acknowledgment throughput, pending commits, and tail latency. Section 6 discusses related work. Section 7 concludes the paper.

2. Background

3. Design

4. Correctness

5. Evaluation

6. Related Work

7. Conclusion

謝辞

参考文献

[1] Bernstein, P. A., Hadzilacos, V. and Goodman, N.: *Concurrency Control and Recovery in Database Systems*, Addison-Wesley (1987).

[2] Papadimitriou, C. H.: The serializability of concurrent database updates, *J. ACM*, Vol. 26, No. 4, p. 631–653 (online), DOI: 10.1145/322154.322158 (1979).

[3] Haerder, T. and Reuter, A.: Principles of transaction-oriented database recovery, *ACM Comput. Surv.*, Vol. 15, No. 4, p. 287–317 (online), DOI: 10.1145/289.291 (1983).

[4] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H. and Schwarz, P.: ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging, *ACM Trans. Database Syst.*, Vol. 17, No. 1, p. 94–162 (online), DOI: 10.1145/128765.128770 (1992).

[5] Tu, S., Zheng, W., Kohler, E., Liskov, B. and Madden, S.: Speedy transactions in multicore in-memory databases, *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, New York, NY, USA, Association for Computing Machinery, p. 18–32 (online), DOI: 10.1145/2517349.2522713 (2013).

[6] Kim, K., Wang, T., Johnson, R. and Pandis, I.: ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1675–1687 (online), DOI: 10.1145/2882903.2882905 (2016).

[7] Yu, X., Pavlo, A., Sanchez, D. and Devadas, S.: TicToc: Time Traveling Optimistic Concurrency Control, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1629–1642 (online), DOI: 10.1145/2882903.2882935 (2016).

[8] Tanabe, T., Hoshino, T., Kawashima, H. and Tatebe, O.: An analysis of concurrency control protocols for in-memory databases with CCBench, *Proc. VLDB Endow.*, Vol. 13, No. 13, p. 3531–3544 (online), DOI: 10.14778/3424573.3424575 (2020).

[9] 孝明神谷，英之川島，喬 星野，修見建部，Komei, K., Hideyuki, K., Takashi, H., Osamu, T.：P-WAL: Paralell Write-ahead Logging, 情報処理学会論文誌データベース（TOD）， Vol. 10, No. 1, pp. 24–39 (online), available from ⟨https://cir.nii.ac.jp/crid/1050282812885062144⟩ (2017).

[10] Xia, Y., Yu, X., Pavlo, A. and Devadas, S.: Taurus: lightweight parallel logging for in-memory database management systems, *Proc. VLDB Endow.*, Vol. 14, No. 2, p. 189–201 (online), DOI: 10.14778/3425879.3425889 (2020).

[11] Haubenschild, M., Sauer, C., Neumann, T. and Leis, V.: Rethinking Logging, Checkpoints, and Recovery for High-Performance Storage Engines, *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, SIGMOD '20, New York, NY, USA, Association for Computing Machinery, p. 877–892 (online), DOI: 10.1145/3318464.3389716 (2020).

[12] Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R. and Sears, R.: Benchmarking cloud serving systems with YCSB, *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, New York, NY, USA, Association for Computing Machinery, p. 143–154 (online), DOI: 10.1145/1807128.1807152 (2010).